



আন্তর্জাতিক ইসলামী বিশ্ববিদ্যালয় চট্টগ্রাম
الجامعة الإسلامية العالمية شيتاغونغ
International Islamic University Chittagong

Department of Computer Science & Engineering

Project Report

Course Title: Computer Algorithms Lab

Course Code: CSE-2422

Project Name: Smart Delivery Route Optimizer

Submitted by:

Member 1 : Jumanah Rahman Anussha (C241472)

Member 2 : Jannatul Ferdous Tania (C241486)

Member 3: Tajia Jahan Tanha(C241502)

Section: 4CF

Submission Date: 20/01/2026

Submitted To:

Miskatul Jannat Tuly

Lecturer, Dept. of CSE, IIUC

Smart Delivery Route Optimizer

Introduction:

Delivery route optimization is the process of determining the most efficient sequence of stops for delivery personnel to minimize travel distance, time, and operational cost while satisfying real-world constraints such as vehicle capacity, service time, and geographic distribution. In practice, this transforms the classical Vehicle Routing Problem (VRP) — an NP-hard challenge in computer science — into a solvable, scalable solution using smart algorithmic strategies.

The Smart Delivery Route Optimizer is an intelligent system that goes beyond basic pathfinding. It dynamically assigns customer addresses to delivery personnel using K-Means Clustering, then selects the most appropriate routing algorithm based on cluster size: exact TSP with Dynamic Programming for tiny sets, Divide & Conquer for medium sets, and Greedy Nearest Neighbor for large sets. Furthermore, it incorporates real-world factors such as weather conditions, round-trip depot return, ASCII-based route visualization, performance competition, and a structured customer feedback mechanism — making it a complete, practical tool for modern logistics.

Objectives:

1. To develop a hybrid algorithm system that adaptively selects optimal routing strategies based on cluster size.
2. To incorporate real-world factors (weather, feedback, visualization) for practical logistics management.

Hardware and Software Requirements:

Hardware Requirements:

- Processor: Intel Core i5 or equivalent
- RAM: 8GB

Software Requirements:

- Operating System: Windows
- Compiler: C++
- IDE: Optional (Code::Blocks)
 - Programming Language: C++
 - Algorithms: K-Means, Greedy, TSP (DP), Divide & Conquer
 - Concepts Used: Data Structures, OOP, Graph Algorithms

Algorithms Used and Their Roles in the System:

Below is a breakdown of each algorithm, its purpose, and where it is applied in the code:

<u>Algorithm</u>	<u>When It Is Used</u>	<u>Why It Is Used</u>
K-Means Clustering	After input collection, before routing	To group geographically close delivery addresses into k clusters (k = number of delivery boys), ensuring each boy serves a compact zone and reducing cross-area travel.
TSP with Dynamic Programming	Inside <code>divideAndConquerRoute()</code> when ≤ 3 addresses	Guarantees the shortest possible route for very small sets (optimal solution). It uses bitmask DP to explore all permutations efficiently.
Divide & Conquer Routing	When cluster size is ≤ 10	Recursively splits addresses by x-coordinate, solves subproblems, and merges routes. Provides near-optimal paths without exponential cost.
Greedy Nearest Neighbor	When cluster size is > 10	At each step, visits the nearest unvisited address. Fast and practical for large sets where optimal solutions are computationally infeasible.
Weather Adjustment Logic	After base route optimization	Applies speed/time multipliers (e.g., $0.7\times$ speed in rain) to reflect real-world delays and improve time estimation accuracy.
Feedback Analysis Engine	In the post-delivery interactive menu	Collects customer ratings, auto-tags issues, and generates performance reports for continuous service improvement.

Algorithm Mapping And Complete System Workflow:

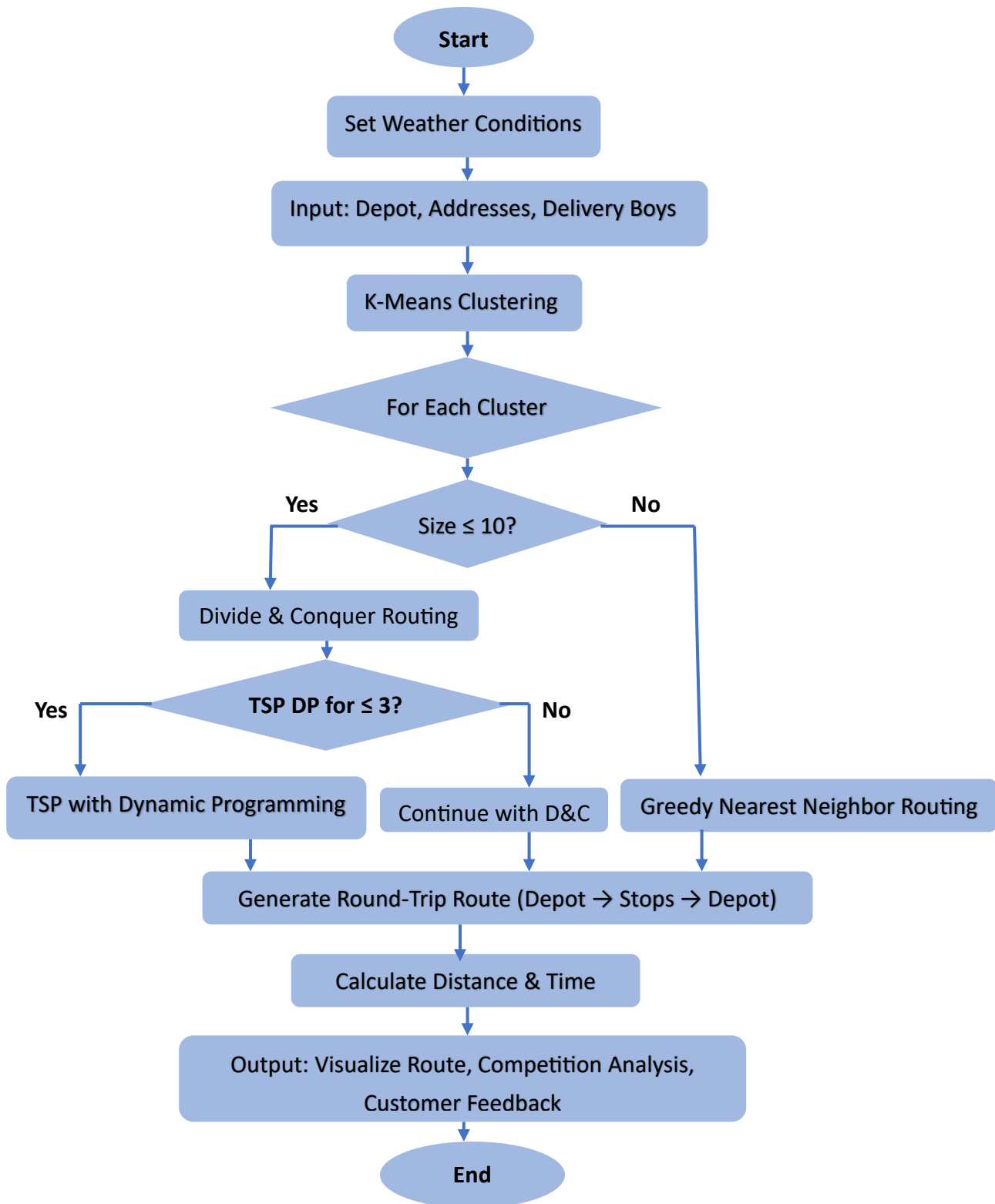
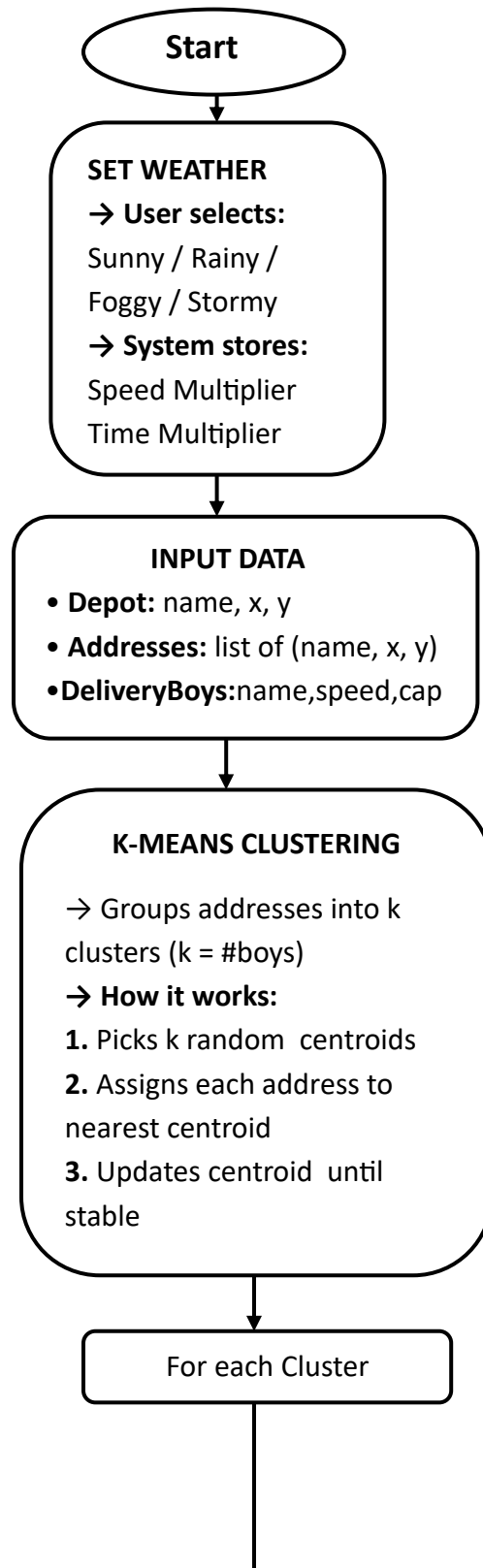
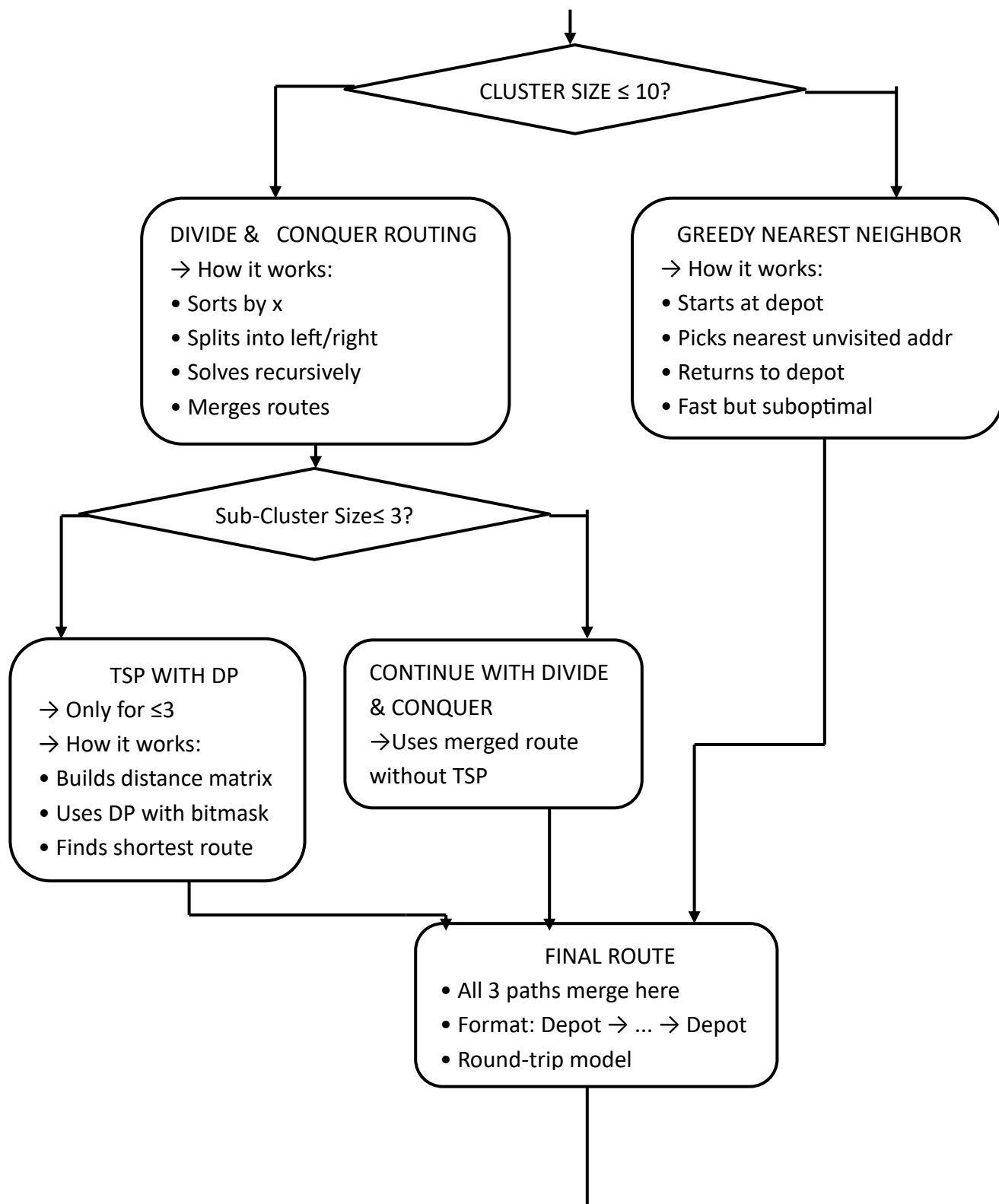


Fig-01: Algorithm Selection Workflow





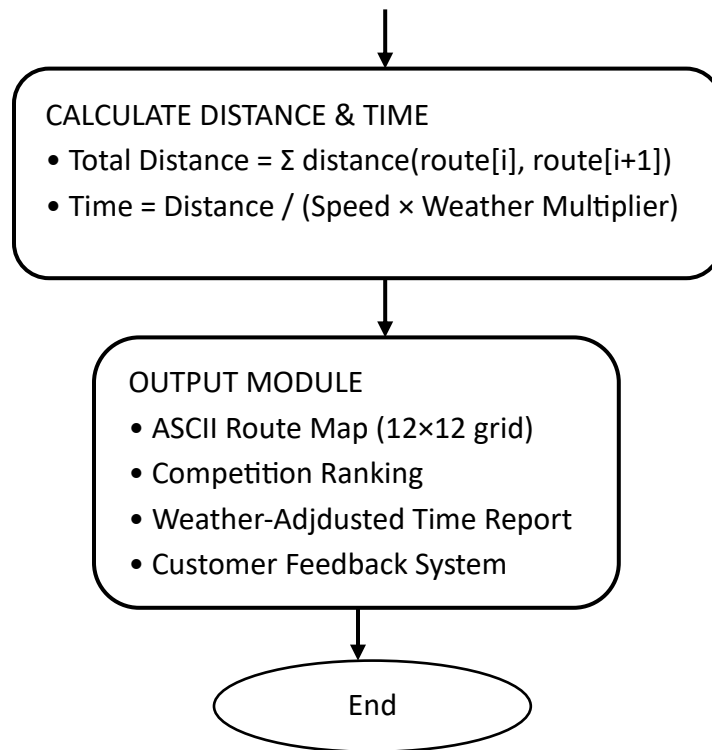


Fig-02 : Smart Delivery Route Optimizer - Main System Workflow

Algorithm Selection Decision Tree:

Addresses → K-Means → Clusters

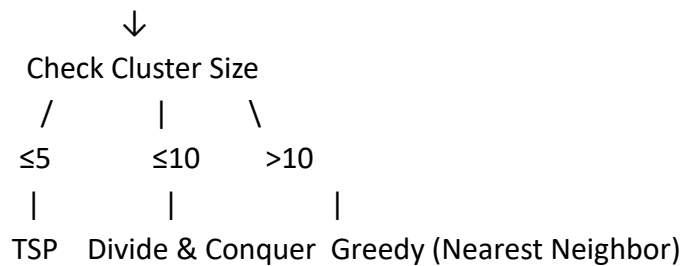


Fig -04: Decision Tree

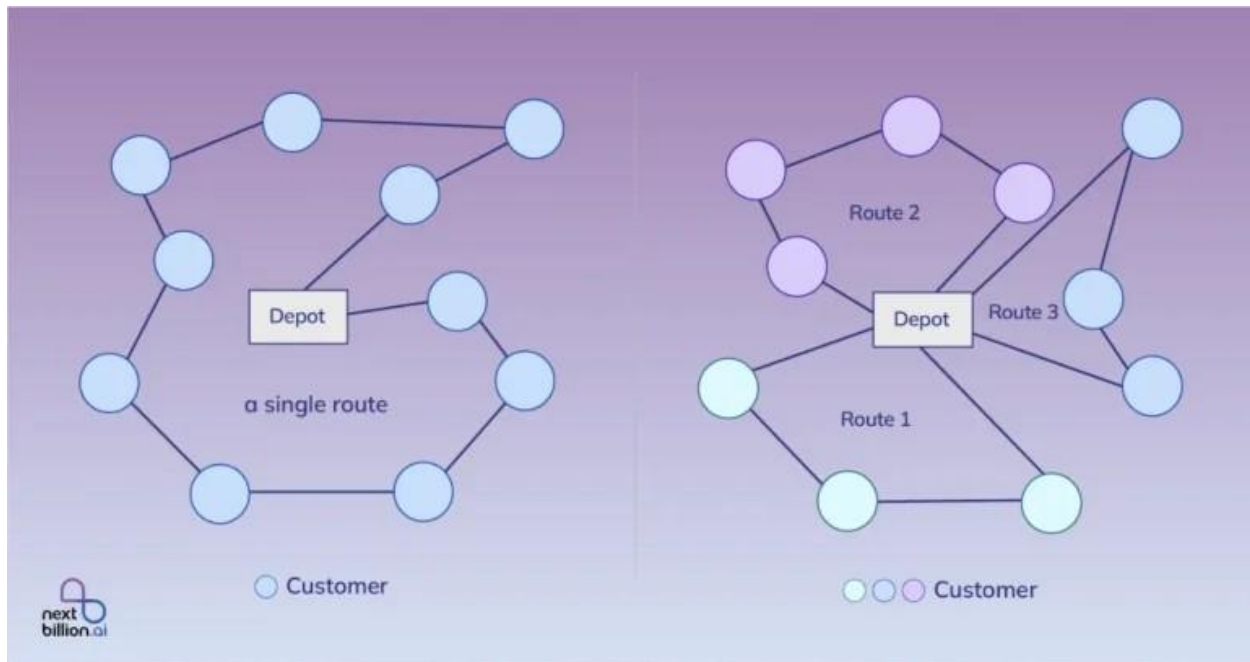


Fig -04: Delivery Execution & Analysis Process

Code:

<https://github.com/taniajannat12/Projects/tree/main/Computer%20Algorithms>

<https://github.com/tajia-aslanbey/C.Algorithm-Project.git>

https://github.com/anussha2023/Computer-Algorithms/tree/main/Algorithm_CSE-2422

Implementation Details:

This C++ program implements a **Delivery Route Optimization System**.

The code:

- Defines structures for **Address**, **Depot**, and **DeliveryBoy**
- Calculates distance between locations using **Euclidean distance**
- Uses **K-Means Clustering** to divide delivery addresses among delivery boys
- Applies **Greedy algorithm** and **Divide & Conquer (TSP-based)** methods to find optimized delivery routes
- Ensures each route **starts and ends at the depot**
- Visualizes routes using a simple grid-based map

- Compares delivery boys based on **total distance and delivery time**
- Adjusts delivery time according to **weather conditions**
- Collects **customer feedback**, calculates ratings, and generates performance reports

The main() function takes all inputs, runs route optimization, applies weather effects, and manages the feedback system.

Output:

```
C:\Users\HP\Desktop\Python' x + v

=====
WELCOME TO DELIVERY ROUTE OPTIMIZATION SYSTEM
=====

=====
WEATHER INFORMATION
=====

Select Current Weather Condition:
1. Sunny/Clear
2. Rainy
3. Foggy
4. Stormy
5. Heatwave
6. Snowy
Enter choice (1-6): 2

Enter Temperature (C): 28
Enter Humidity (%): 80
Enter Wind Speed (km/h): 15

=====
CURRENT WEATHER REPORT
=====

WEATHER CONDITIONS:
- Condition: Rainy weather
- Temperature: 28C
- Humidity: 80%
- Wind Speed: 15 km/h
```

```
C:\Users\HP\Desktop\Python' x + v

Enter Wind Speed (km/h): 15

=====
CURRENT WEATHER REPORT
=====

WEATHER CONDITIONS:
- Condition: Rainy weather
- Temperature: 28C
- Humidity: 80%
- Wind Speed: 15 km/h

DELIVERY IMPACT:
- Speed Multiplier: 0.7x
- Time Multiplier: 1.4x

SAFETY TIPS:
Use windshield wipers, maintain safe distance
=====

=====
DEPOT INFORMATION
=====

Enter Depot Name: maindepot
Enter Depot X Coordinate: 0
Enter Depot Y Coordinate: 0

=====
ADDRESS INFORMATION
=====

Number of Delivery Addresses: 3
```



```
C:\Users\HP\Desktop\Python' X + v

=====
TOTAL DISTANCE: 12.31 km
=====

=====
COMPETITION RESULTS
=====

INDIVIDUAL PERFORMANCE:
=====
john      | Distance: 8.25   km | Speed: 20.00 km/h | Time: 24.7   min
alex      | Distance: 12.31  km | Speed: 15.00 km/h | Time: 49.2   min
=====

FINAL RANKING
=====
1. john      - 24.7 minutes
2. alex      - 49.2 minutes
=====

WINNER: john returns to depot first!

TIME GAPS BETWEEN DELIVERY BOYS:
alex is 24.5 minutes behind john

=====

=====
DELIVERY OPTIMIZATION COMPLETED
=====

=====
WEATHER-ADJUSTED ANALYSIS
=====

Delivery times adjusted for weather conditions:
john: 30.0 min -> 42.0 min (40% change)
```

```
C:\Users\HP\Desktop\Python' X + v

=====
CUSTOMER FEEDBACK FORM
=====

Customer Name: jhon
Phone Number: 124414214
Delivery Address: c
Delivery Boy Name: shopA
Delivery Boy ID: 1

Rating (1-5 stars): 4
Comments (optional): no

Was there any issue with the delivery? (1=Yes, 0=No): 0

REVIEW ADDED SUCCESSFULLY!
Customer: jhon
Rating: 4/5

=====
CUSTOMER FEEDBACK SYSTEM
=====

1. Submit Customer Feedback
2. View Delivery Boy Performance
3. Show Overall Summary
4. Exit

Enter your choice: 4

Thank you for using the system!

=====
SYSTEM SUMMARY REPORT
=====

DELIVERY OPERATION COMPLETED
- Total Addresses: 3
- Delivery Personnel: 2
- Depot: maindepot
- Weather Condition: Rainy
- Weather Impact: 40% time increase

SYSTEM READY FOR NEXT DELIVERY CYCLE

=====

Process returned 0 (0x0)   execution time : 388.187 s
Press any key to continue.
|
```

Output Analysis:

Distance Calculations:

Distances are calculated using Euclidean formula:

Distance = $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

John’s Route:

- Depot → shopC = 4.12 km
- shopC → Depot = 4.12 km
- Total Distance = 8.25 km

Alex’s Route:

- Depot → shopA = 3.61 km
- shopA → shopB = 3.61 km
- shopB → Depot = 5.10 km
- Total Distance = 12.31 km

2 Delivery Time Calculation

Base time = Total distance $\div$ Speed $\times$ 60 (minutes)

John: 24.7 min = $8.25/20 \times 60$

Alex: 49.2 min = $12.31/15 \times 60$

3 Weather-Adjusted Time

Weather multiplier (Rainy) = $1.4 \times$ base time

John: $24.7 \times 1.4 \approx 42.0$ min

Alex: $49.2 \times 1.4 \approx 56.0$ min

Impact: 40% increase in delivery time due to rain.

4 Time Gap Between Delivery Boys

Time Gap = $49.2 - 24.7 = 24.5$ min

5 Customer Feedback Example

- Customer gave 4/5 rating for “shopA” delivery.
- No issues reported.

Delivery Operation Summary

- Total Addresses: 3
- Delivery Personnel: 2 (John, Alex)
- Depot: maindepot
- Total Distances: John = 8.25 km, Alex = 12.31 km
- Base Delivery Times: John = 24.7 min, Alex = 49.2 min
- Weather-Adjusted Times: John = 42.0 min, Alex = 56.0 min (+40%)
- Time Gap: 24.5 min (Alex behind John)
- Winner: John – returned to depot first
- Customer Feedback: 1 review, rating 4/5, no issues reported

Complexity Analysis:

The system uses multiple algorithmic techniques such as K-Means clustering, Greedy heuristics, Divide & Conquer, and Dynamic Programming. Each algorithm is applied based on input size to maintain efficiency and scalability.

Distance Calculation: `double distance(const Address &a, const Address &b)`

(The distance between two points is computed using a fixed number of arithmetic operations.)

Time Complexity: $O(1)$

Space Complexity: $O(1)$

K-Means Clustering: `kMeansClustering(addresses, k)`

(Assign each address to nearest centroid $\rightarrow O(nk)$

Update centroids $\rightarrow O(n)$

Repeated for max $l = 100$ iterations (constant))

Time Complexity: $(l \times n \times k) \approx O(nk)$

Space Complexity: $O(n+k)$

Greedy Route Optimization (Nearest Neighbor): `greedyRoute(addresses, start)`

(For each address, find nearest unvisited address)

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

TSP using Dynamic Programming (Bitmask DP): `tspDP(distMatrix, start)`

(The algorithm evaluates all possible subsets of nodes using bitmask-based dynamic programming.)

Time Complexity: $O(n^2 \times 2^n)$

Space Complexity: $O(n \times 2^n)$

Divide & Conquer Route Optimization: `divideAndConquerRoute(addresses, start)`

(The address list is recursively divided into smaller parts and merged after sorting.

Recurrence Relation: $T(n) = 2T(n/2) + O(n)$)

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

Route Visualization: `visualizeRoute(route)`

(Grid mapping + distance calculations)

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Competition Analysis : analyzeCompetition(routes, boys)

(Distance calculation per route)

Time Complexity: $O(n)$

Space Complexity: $O(k)$

Weather Adjustment System : WeatherManager

(Weather impact calculations involve only constant-time operations.)

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Customer Feedback System :

(Feedback data is stored and sorted to generate performance summaries.)

Time Complexity: $O(r \log r)$

Space Complexity: $O(r)$

Complexity Table:

Module	Time Complexity	Space Complexity
Distance Calculation	$O(1)$	$O(1)$
K-Means Clustering	$O(nk)$	$O(n + k)$
Greedy Route Optimization	$O(n^2)$	$O(n)$
TSP (Dynamic Programming)	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$
Divide & Conquer Routing	$O(n \log n)$	$O(n)$
Visualization	$O(n)$	$O(1)$
Weather System	$O(1)$	$O(1)$
Feedback System	$O(r \log r)$	$O(r)$

Worst-Case Time Complexity:

$$O(nk + n^2 + n \log n)$$

(where n is the number of delivery addresses and k is the number of delivery persons.)

Worst-Case Space Complexity:

$$O(n + k + r)$$

(where n is the number of delivery addresses, k is the number of delivery persons, and r is the number of customer reviews.)

TIME COMPLEXITY:

BEST: $O(n \log n)$

WORST: $O(n^2)$

AVERAGE: $O(n \log n)$

SPACE COMPLEXITY:

BEST: $O(n)$ (Small clusters, no TSP)

WORST: $O(n \cdot 2^n)$ (TSP for $n \leq 5$, practical limit)

AVERAGE: $O(n + k)$ (typical case)

Overall Time Complexity:

$$O(n^2)$$

Moderate time complexity. Works well for typical delivery scenarios (≤ 1000 addresses) but may need optimization for city-scale operations.

Overall Space Complexity:

$$O(n)$$

"Efficient memory usage. Linear growth ensures the system can handle increasing addresses without memory issues."

"A practical delivery system with efficient memory usage and acceptable time complexity for real-world medium-scale operations. Could benefit from algorithmic improvements for large scale deployment."

Challenges Faced And Future Scope:

Advantages

- Optimizes delivery routes efficiently
- Reduces total distance and delivery time
- Manages multiple delivery persons easily
- Adjusts delivery time based on weather conditions
- Evaluates performance using customer feedback

Limitations:

- Does not use real road maps
- Traffic conditions are not considered
- Performance may decrease for large datasets
- No real-time or internet-based data support
- **Delivery output times are sometimes inaccurate**
- **Route visualization does not always display correctly;** proper map visualization will be implemented in future

Future Improvements:

- Integration with Google Maps or real-world maps
- Include real-time traffic information
- Develop a mobile application
- Use AI-based advanced route optimization
- Add map integration for real navigation and live traffic updates for accurate delivery time predictions
- Fix output time calculations and improve route visualization

Conclusion:

The Delivery Route Optimization System efficiently manages multiple delivery personnel, optimizes routes to reduce total distance and time, and adjusts delivery estimates based on weather conditions. While the system currently lacks real road maps, traffic data, and accurate route visualization, it provides a strong foundation for future improvements such as integration with real-world maps, AI-based route optimization, and real-time traffic updates. Overall, this project demonstrates the potential of combining route optimization algorithms, clustering, and customer feedback to improve delivery efficiency and performance.

References:

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. – K-Means, TSP, greedy algorithms, algorithm complexity.
- Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Pearson. – Divide-and-conquer and clustering strategies.
- MacQueen, J. (1967). Some Methods for Classification and Analysis of Multivariate Observations. – Original K-Means algorithm.
- Lawler, E. L., et al. (1985). *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley. – TSP dynamic programming.
- GeeksforGeeks: K-Means Clustering in C++ – <https://www.geeksforgeeks.org/k-means-clustering/>
- GeeksforGeeks: Traveling Salesman Problem using DP – <https://www.geeksforgeeks.org/dsa/travelling-salesman-problem-using-dynamic-programming/>